Agent Swarm - Nodos Dinamicos (Auto-Discovery)

Fecha: 2026-03-21 **Autor:** Cesar / Claude Code **Estado:** Diseno de arquitectura **Principio:** Zero-config. Una maquina nueva se conecta y empieza a trabajar.

---

1. Principio Fundamental

NO existe una lista hardcodeada de maquinas. El sistema se adapta a N nodos:

```
1 maquina  → trabaja sola, toma todas las tareas
2 maquinas → se dividen el trabajo por capacidades
5 maquinas → se reparten automaticamente
10 maquinas → escala sin cambiar una sola linea de codigo
```

Cada maquina es un **nodo** que:

1. Se registra al iniciar (auto-announce)
2. Declara sus **capacidades** (que sabe hacer)
3. Descubre a los otros nodos via DynamoDB
4. Toma tareas compatibles con sus capacidades
5. Si se apaga, desaparece automaticamente (TTL)

---

2. Identidad de Nodo

2.1 Archivo de identidad local: `.covacha-node.yml`

Cada maquina tiene UN archivo en su home directory que la identifica:

```
# ~/.covacha-node.yml
node_id: "cesar-macbook-pro"      # Unico, human-readable
capabilities:
  - backend                        # Puede trabajar en repos backend
  - frontend                       # Puede trabajar en repos frontend
  - testing                        # Puede ejecutar tests
repos:                             # Repos clonados localmente
  - path: /Users/casp/sandboxes/superpago/covacha-payment
    name: covacha-payment
  - path: /Users/casp/sandboxes/superpago/covacha-core
    name: covacha-core
  - path: /Users/casp/sandboxes/superpago/mf-core
    name: mf-core
  - path: /Users/casp/sandboxes/superpago/covacha-projects
    name: covacha-projects
roles:                             # Roles de equipo que puede asumir
  - developer
  - tech_lead
```

```
max_concurrent_tasks: 1            # Cuantas tareas a la vez (default 1
preferred_model: "sonnet"          # Modelo Claude por default
```

2.2 Auto-generacion del node_id

Si no existe `.covacha-node.yml`, el sistema lo genera automaticamente:

```
import platform
import hashlib

def generate_node_id() -> str:
    """Genera un node_id unico basado en hostname + username."""
    hostname = platform.node()          # "Cesars-MacBook-Pro.local"
    username = os.getenv("USER", "unknown")
    # Limpieza
    clean_host = hostname.split(".")[0].lower().replace(" ", "-")
    return f"{username}-{clean_host}"   # "casp-cesars-macbook-pro"
```

2.3 Auto-deteccion de repos locales

```
import glob

def detect_local_repos(base_path: str = "~/sandboxes/superpago") -> lis
    """Detecta repos clonados buscando directorios con .git"""
    repos = []
    for git_dir in glob.glob(f"{base_path}/*/.git"):
        repo_path = os.path.dirname(git_dir)
        repo_name = os.path.basename(repo_path)
        repos.append({"path": repo_path, "name": repo_name})
    return repos
```

2.4 Auto-deteccion de capabilities

```
def detect_capabilities(repos: list[dict]) -> list[str]:
    """Infiere capabilities a partir de los repos clonados."""
    caps = set()
    for repo in repos:
        name = repo["name"]
        if name.startswith("covacha-"):
            caps.add("backend")
        if name.startswith("mf-"):
            caps.add("frontend")
        if "test" in name or "qa" in name:
            caps.add("testing")
        if name == "covacha-projects":
            caps.add("ops")
            caps.add("project_owner")
    # Si tiene pytest instalado → puede testear
    if shutil.which("pytest"):
        caps.add("testing")
    # Si tiene ng instalado → puede hacer frontend
    if shutil.which("ng"):
        caps.add("frontend")
```

```
    return list(caps)
```

---

3. Registro en DynamoDB

3.1 Nuevo prefijo: NODE#

```
{
  "PK": "NODE#cesar-macbook-pro",
  "SK": "STATUS",
  "node_id": "cesar-macbook-pro",
  "capabilities": ["backend", "frontend", "testing"],
  "repos": [
    {"path": "/Users/casp/.../covacha-payment", "name": "covacha-paymen
    {"path": "/Users/casp/.../mf-core", "name": "mf-core"}
  ],
  "roles": ["developer", "tech_lead"],
  "max_concurrent_tasks": 1,
  "preferred_model": "sonnet",
  "current_task": null,
  "status": "idle",
  "last_heartbeat": 1711036800,
  "registered_at": 1711033200,
  "version": "1.0.0",
  "os": "darwin",
  "ttl": 1711123200
}
```

TTL: 24 horas desde el ultimo heartbeat. Si la maquina no hace ping en 24h, DynamoDB la borra automaticamente. Al volver a encenderse, se re-registra.

3.2 Registro (al iniciar sesion)

```
def register_node(node_config: dict) -> None:
    """Registra o actualiza este nodo en DynamoDB."""
    table = get_dynamo_client()
    now = int(time.time())
    table.put_item(Item={
        "PK": f"NODE#{node_config['node_id']}",
        "SK": "STATUS",
        "node_id": node_config["node_id"],
        "capabilities": node_config["capabilities"],
        "repos": node_config["repos"],
        "roles": node_config.get("roles", ["developer"]),
        "max_concurrent_tasks": node_config.get("max_concurrent_tasks",
        "preferred_model": node_config.get("preferred_model", "sonnet")
        "current_task": None,
        "status": "idle",
        "last_heartbeat": now,
        "registered_at": now,
        "version": "1.0.0",
        "os": platform.system().lower(),
```

```
        "ttl": now + 86400,  # 24h
    })
```

3.3 Heartbeat (cada 2 min)

```
def heartbeat(node_id: str) -> None:
    """Actualiza last_heartbeat y renueva TTL."""
    table = get_dynamo_client()
    now = int(time.time())
    table.update_item(
        Key={"PK": f"NODE#{node_id}", "SK": "STATUS"},
        UpdateExpression="SET last_heartbeat = :now, #ttl = :ttl",
        ExpressionAttributeNames={"#ttl": "ttl"},
        ExpressionAttributeValues={
            ":now": now,
            ":ttl": now + 86400,
        },
    )
```

3.4 Discovery (ver todos los nodos)

```
def discover_nodes(only_active: bool = True) -> list[dict]:
    """Descubre todos los nodos registrados en el swarm."""
    table = get_dynamo_client()
    nodes = _paginate_scan(table, Attr("SK").eq("STATUS") & Attr("PK").

    if only_active:
        now = int(time.time())
        stale_threshold = 300  # 5 min sin heartbeat = stale
        nodes = [
            n for n in nodes
            if (now - n.get("last_heartbeat", 0)) < stale_threshold
        ]

    return nodes


def get_swarm_summary() -> dict:
    """Resumen del swarm: cuantos nodos, que capabilities, quien trabaj
    nodes = discover_nodes(only_active=True)
    return {
        "total_nodes": len(nodes),
        "idle_nodes": [n for n in nodes if n["status"] == "idle"],
        "busy_nodes": [n for n in nodes if n["status"] == "working"],
        "capabilities": _aggregate_capabilities(nodes),
        "coverage": _calculate_coverage(nodes),
    }
```

---

4. Asignacion Dinamica de Tareas

4.1 Compatibilidad tarea ↔ nodo

Una tarea es compatible con un nodo si:

```
def is_task_compatible(task: dict, node: dict) -> bool:
    """Verifica si un nodo puede trabajar en una tarea."""

    # 1. El nodo tiene el repo clonado localmente
    task_repo = task.get("repo", "")
    node_repos = [r["name"] for r in node.get("repos", [])]
    if task_repo and task_repo not in node_repos:
        return False

    # 2. Al menos una capability del nodo matchea un label de la tarea
    task_labels = set(task.get("labels", []))
    node_caps = set(node.get("capabilities", []))
    if not task_labels.intersection(node_caps):
        return False

    # 3. El nodo no esta al maximo de tareas concurrentes
    if node.get("status") == "working":
        current_count = node.get("current_task_count", 1)
        if current_count >= node.get("max_concurrent_tasks", 1):
            return False

    # 4. Dependencias resueltas
    # (delegado a is_task_unblocked)

    return True
```

4.2 Scoring de asignacion

Cuando multiples nodos pueden tomar la misma tarea, se usa scoring:

```
def score_assignment(task: dict, node: dict) -> float:
    """Score de 0.0 a 1.0 — que tan buena es esta asignacion."""
    score = 0.0

    # Capability match (0.0 - 0.4)
    # Mas labels en comun = mejor match
    task_labels = set(task.get("labels", []))
    node_caps = set(node.get("capabilities", []))
    overlap = len(task_labels.intersection(node_caps))
    total = len(task_labels) or 1
    score += (overlap / total) * 0.4

    # Repo locality (0.0 - 0.3)
    # El nodo tiene el repo clonado = bonus
    task_repo = task.get("repo", "")
    node_repos = [r["name"] for r in node.get("repos", [])]
    if task_repo in node_repos:
        score += 0.3

    # Idle time (0.0 - 0.15)
    # Nodo que lleva mas tiempo idle tiene prioridad
```

```
    if node["status"] == "idle":
        idle_seconds = int(time.time()) - node.get("last_heartbeat", 0)
        score += min(idle_seconds / 600, 1.0) * 0.15  # max a los 10 mi

    # Learnings bonus (0.0 - 0.15)
    # Si el nodo tiene learnings del mismo repo, conoce el contexto
    learnings = get_learnings(task_repo)
    if learnings.get("gotchas"):
        score += 0.15

    return round(score, 3)
```

4.3 find_next_task dinamico

```
def find_next_task(node_id: str) -> dict | None:
    """Encuentra la mejor tarea para este nodo especifico."""

    # 1. Obtener info del nodo
    node = get_node(node_id)
    if not node:
        raise ValueError(f"Nodo {node_id} no registrado")

    # 2. Obtener tareas disponibles
    available = get_available_tasks(status="todo")

    # 3. Filtrar por compatibilidad
    compatible = [t for t in available if is_task_compatible(t, node)]

    # 4. Filtrar por dependencias resueltas
    unblocked = [t for t in compatible if is_task_unblocked(t["number"]

    # 5. Ordenar por score
    scored = [(t, score_assignment(t, node)) for t in unblocked]
    scored.sort(key=lambda x: x[1], reverse=True)

    return scored[0][0] if scored else None
```

---

5. Roles Dinamicos

5.1 Asignacion automatica de roles

Los roles NO son fijos por maquina. Se asignan dinamicamente segun las capabilities y el estado del swarm:

```
ROLE_REQUIREMENTS = {
    "tech_lead": {
        "required_caps": ["backend", "architecture"],
        "preferred_caps": ["security"],
        "max_per_swarm": 1,  # Solo 1 tech lead a la vez
    },
    "developer": {
```

```
        "required_caps": [],  # Cualquier nodo puede ser developer
        "preferred_caps": ["backend", "frontend"],
        "max_per_swarm": None,  # Sin limite
    },
    "tester": {
        "required_caps": ["testing"],
        "preferred_caps": ["e2e"],
        "max_per_swarm": None,
    },
    "project_owner": {
        "required_caps": ["ops"],
        "preferred_caps": ["project_owner"],
        "max_per_swarm": 1,
    },
    "reviewer": {
        "required_caps": ["backend"],  # o frontend
        "preferred_caps": [],
        "max_per_swarm": None,
    },
}


def assign_roles(nodes: list[dict]) -> dict[str, str]:
    """Asigna roles a nodos dinamicamente segun capabilities del swarm.

    Retorna: {node_id: role_asignado}
    """
    assignments = {}
    available_nodes = list(nodes)

    # 1. Primero asignar roles con max_per_swarm (escasos)
    for role, reqs in sorted(
        ROLE_REQUIREMENTS.items(),
        key=lambda x: (x[1]["max_per_swarm"] or 999)
    ):
        if reqs["max_per_swarm"] and role_count(assignments, role) >= r
            continue

        # Encontrar mejor nodo para este rol
        candidates = [
            n for n in available_nodes
            if all(cap in n["capabilities"] for cap in reqs["required_c
        ]

        if candidates:
            # Preferir nodos que declaran este rol en su config
            best = max(candidates, key=lambda n: (
                role in n.get("roles", []),
                len(set(n["capabilities"]).intersection(reqs["preferred
            ))
            assignments[best["node_id"]] = role
            if reqs["max_per_swarm"] == 1:
                available_nodes.remove(best)
```

```
    # 2. Nodos sin rol asignado → developer (default)
    for node in available_nodes:
        if node["node_id"] not in assignments:
            assignments[node["node_id"]] = "developer"

    return assignments
```

5.2 Escenarios de escalamiento

```
┌──────────────────────────────────────────────────────────────┐
│                  1 NODO (minimo viable)                      │
│                                                              │
│  cesar-macbook: developer + tester + project_owner           │
│  → Hace todo: implementa, testea, acepta tickets             │
│  → Auto-evaluacion: pre_release + qa_review en el mismo nodo │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│                  2 NODOS (equipo minimo)                     │
│                                                              │
│  nodo-1 [backend,frontend]: developer + tech_lead            │
│  nodo-2 [testing,ops]:      tester + project_owner           │
│                                                              │
│  → nodo-1 implementa, nodo-2 evalua y acepta                 │
│  → Separacion de concerns: quien implementa ≠ quien valida   │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│                  3 NODOS (equipo balanceado)                 │
│                                                              │
│  nodo-1 [backend]:    tech_lead + developer                  │
│  nodo-2 [frontend]:   developer                              │
│  nodo-3 [testing,ops]: tester + project_owner                │
│                                                              │
│  → Backend y frontend en paralelo                            │
│  → nodo-3 como QA/PO dedicado                                │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│                  5 NODOS (equipo completo)                   │
│                                                              │
│  nodo-1 [backend,arch]:  tech_lead                           │
│  nodo-2 [frontend]:      developer                           │
│  nodo-3 [backend]:       developer                           │
│  nodo-4 [testing]:       tester                              │
│  nodo-5 [ops]:           project_owner                       │
│                                                              │
│  → Especializacion completa                                  │
│  → 2 backends en paralelo                                    │
│  → QA y PO dedicados                                         │
└──────────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────┐
│                  10 NODOS (equipo grande)                    │
│                                                              │
│  nodo-1:  tech_lead (backend arch)                           │
│  nodo-2:  developer (frontend core)                          │
│  nodo-3:  developer (backend payments)                       │
│  nodo-4:  developer (backend notifications)                  │
│  nodo-5:  developer (frontend marketing)                     │
│  nodo-6:  developer (frontend dashboard)                     │
│  nodo-7:  tester (backend QA)                                │
│  nodo-8:  tester (frontend QA + E2E)                         │
│  nodo-9:  project_owner                                      │
│  nodo-10: reviewer (code review dedicado)                    │
│                                                              │
│  → Maxima paralelizacion                                     │
│  → Multiples testers para no ser bottleneck                  │
└──────────────────────────────────────────────────────────────┘
```

---

6. Ciclo de Vida de un Nodo

```
┌──────────────────────────────────────────────────────────────┐
│                   CICLO DE VIDA DE UN NODO                   │
│                                                              │
│   ┌──────────┐                                               │
│   │  NUEVO   │  Maquina nueva, no tiene .covacha-node.yml    │
│   └──────────┘                                               │
│        │                                                     │
│        ▼                                                     │
│   ┌──────────┐                                               │
│   │  SETUP   │  covacha init                                 │
│   │          │  → Detecta repos locales                      │
│   │          │  → Infiere capabilities                       │
│   │          │  → Genera .covacha-node.yml                   │
│   │          │  → Pregunta al usuario: "Confirmar caps?"     │
│   └──────────┘                                               │
│        │                                                     │
│        ▼                                                     │
│   ┌──────────┐                                               │
│   │ REGISTER │  register_node() → DynamoDB NODE#             │
│   │          │  → Descubre otros nodos                       │
│   │          │  → Se le asigna rol automaticamente           │
│   │          │  → MSG broadcast: "Nuevo nodo: X (caps: ...)" │
│   └──────────┘                                               │
│        │                                                     │
│        ▼                                                     │
│   ┌──────────┐                                               │
│   │  ACTIVE  │  Agent Loop corriendo                         │
│   │          │  → Heartbeat cada 2 min                       │
│   │          │  → Toma tareas, implementa, evalua            │
│   │          │  → Comunica con otros nodos via MSG#          │
│   └──────────┘                                               │
│        │                                                     │
│        │                                                     │
│        ├─── Shutdown graceful ──┐                            │
│        │    (Ctrl+C o apagado)  ▼                            │
```

```
│       │                     ┌──────────┐                        │
│       │                     │ LEAVING  │                        │
│       │                     │ → Release task actual             │
│       │                     │ → MSG: "Nodo X se fue"           │
│       │                     │ → Status: "offline"              │
│       │                     └──────────┘                        │
│       │                                                         │
│       └─── Crash / desconexion ──┐                              │
│                                  ▼                              │
│                             ┌──────────┐                        │
│                             │  STALE   │                        │
│                             │ (5 min sin heartbeat)             │
│                             │ → Otros nodos detectan            │
│                             │ → Auto-release de su task         │
│                             │ → MSG: "Nodo X stale"             │
│                             └──────────┘                        │
│                                  ▼                              │
│                             ┌──────────┐                        │
│                             │ EXPIRED  │                        │
│                             │ (24h TTL)                         │
│                             │ → DynamoDB borra NODE#            │
│                             │ → Desaparece del swarm            │
│                             └──────────┘                        │
│                                                                 │
│                                                                 │
│   ┌──────────┐                                                  │
│   │ REJOIN   │  Maquina vuelve a encenderse                     │
│   │          │  → Lee .covacha-node.yml existente               │
│   │          │  → Re-register con mismo node_id                 │
│   │          │  → Recupera learnings del modulo                 │
│   │          │  → MSG: "Nodo X volvio"                          │
│   │          │  → Continua donde se quedo                       │
│   └──────────┘                                                  │
└─────────────────────────────────────────────────────────────────┘
```

---

7. CLI: `covacha` (comando principal)

7.1 Comandos

```
# Setup inicial (primera vez en una maquina nueva)
covacha init
  → Detecta repos, infiere caps, genera .covacha-node.yml
  → Registra nodo en DynamoDB
  → Instala MCP servers (Context Mode + Covacha MCP)

# Iniciar el loop autonomo
covacha start
  → Lee .covacha-node.yml
  → register_node()
  → Inicia Agent Loop

# Ver estado del swarm
```

```
covacha status
  → Descubre todos los nodos activos
  → Muestra: node_id | role | status | current_task | last_seen

# Ver nodos del swarm
covacha nodes
  → Lista todos los nodos con capabilities y repos

# Detener gracefully
covacha stop
  → Release task actual
  → MSG broadcast "leaving"
  → Status: offline

# Agregar capability manual
covacha cap add testing
  → Actualiza .covacha-node.yml + DynamoDB

# Agregar repo manual
covacha repo add /path/to/new-repo
  → Detecta nombre, actualiza .covacha-node.yml + DynamoDB

# Forzar re-discovery
covacha discover
  → Re-escanea repos locales
  → Re-infiere capabilities
  → Actualiza .covacha-node.yml + DynamoDB
```

7.2 Ejemplo: agregar una maquina nueva

```
# En la maquina nueva (Mac Mini recien comprada):

# 1. Clonar repos que va a trabajar
cd ~/sandboxes/superpago
git clone git@github.com:baatdigital/covacha-payment.git
git clone git@github.com:baatdigital/covacha-core.git

# 2. Instalar Claude Code + Context Mode
brew install claude-code
# /plugin marketplace add mksglu/context-mode
# /plugin install context-mode@context-mode

# 3. Instalar Covacha MCP Server
npm install -g covacha-mcp-server

# 4. Inicializar nodo
covacha init

# Output:
# ┌──────────────────────────────────────────┐
# │ Covacha Agent Swarm - Node Setup         │
# │                                          │
# │ Node ID: casp-mac-mini-m4                │
```

```
# │                                          │
# │ Repos detectados:                        │
# │   ✓ covacha-payment                      │
# │   ✓ covacha-core                         │
# │                                          │
# │ Capabilities inferidas:                  │
# │   ✓ backend (covacha-* repos)            │
# │   ✓ testing (pytest disponible)          │
# │                                          │
# │ Rol asignado: developer                  │
# │                                          │
# │ Swarm actual:                            │
# │   cesar-macbook-pro [tech_lead] WORKING  │
# │   casp-mac-studio   [tester]    IDLE     │
# │   → Tu (casp-mac-mini-m4) se une como   │
# │     developer (backend)                  │
# │                                          │
# │ Confirmar? [Y/n]                         │
# └──────────────────────────────────────────┘

# 5. Iniciar loop
covacha start

# Output:
# ✓ Nodo registrado en swarm (3 nodos activos)
# ✓ MSG broadcast: "casp-mac-mini-m4 se unio (caps: backend, testing)"
# ✓ Buscando tarea compatible...
# ✓ Tarea encontrada: ISS-048 "Agregar webhook de SPEI" (score: 0.87)
# ✓ Claiming ISS-048...
# → Iniciando sesion Claude Code...
```

---

8. Resiliencia

8.1 Nodo se cae (crash)

```
T=0:00  nodo-3 crashea (kernel panic, se apaga la luz, etc.)
T=0:02  No hay heartbeat de nodo-3
T=0:05  Otros nodos detectan: nodo-3 stale
        → check_stale_nodes() en el heartbeat de cada nodo
T=0:05  Auto-release de ISS-045 (task de nodo-3)
        → ISS-045 vuelve a status "todo"
        → MSG: "nodo-3 stale, ISS-045 re-encolada"
T=0:06  Nodo con mejor score para ISS-045 la reclama
        → Puede ser otro nodo o puede esperar

T=1:00  nodo-3 se reinicia
        → covacha start (o auto-start via launchd/systemd)
        → Re-register con mismo node_id
        → MSG: "nodo-3 volvio"
        → Busca nueva tarea (ISS-045 ya fue reclamada por otro)
```

8.2 Swarm se queda con 1 nodo

```
Si todos los nodos se caen excepto 1:
→ El nodo restante asume TODOS los roles (developer + tester + PO)
→ Auto-evaluacion hace todo el pipeline en el mismo nodo
→ No pierde funcionalidad, solo velocidad

Cuando los otros nodos vuelven:
→ Re-register → roles se redistribuyen automaticamente
→ Tareas se reparten segun capabilities
```

8.3 Nodo nuevo se une mid-sprint

```
Sprint en progreso, 2 nodos trabajando.
Se compra Mac Mini nueva y se une:

→ covacha init → detecta repos → register
→ discover_nodes() → ve 2 nodos activos
→ assign_roles() → le asigna "developer" (o "tester" si faltan caps)
→ find_next_task() → busca tareas compatibles que nadie este haciendo
→ Empieza a contribuir inmediatamente
→ Los otros nodos lo ven en su proximo heartbeat
→ MSG: "nuevo nodo unido al swarm"
```

---

9. DynamoDB Schema Actualizado

```
covacha-agent-memory (single-table, PAY_PER_REQUEST)
│
├── NODE#{node_id}│STATUS          [NUEVO]
│   → node_id, capabilities, repos, roles, status,
│     current_task, last_heartbeat, registered_at, ttl
│
├── TASK#{number}│META
│   → number, title, labels, status, repo, branch,
│     github_node_id, recommended_model
│
├── TASK#{number}│LOCK
│   → locked_by (node_id), locked_at, ttl
│
├── DEP#{blocked}│DEPENDS_ON#{blocker}
│   → blocker_task, blocker_status, blocked_task
│
├── MSG#{timestamp}-{node_id}│BROADCAST
│   → from_node, type, task_id, payload, read_by[], ttl
│
├── CONTRACT#{name}-{version}│META
│   → method, path, request_schema, response_schema,
│     published_by (node_id), task_id
│
├── LEARNING#{module}│patterns
│   → gotchas[], updated_by (node_id), updated_at
```

```
│
└── SYNC#github│META
    → last_sync, errors[]
```

Cambios clave:

- `TEAM#` se reemplaza por `NODE#` (mas descriptivo)
- `locked_by` ahora es `node_id` (no "team")
- `MSG#` usa `node_id` en vez de "machine"
- Todo referencia a nodos, no a maquinas hardcodeadas

---

10. Migracion desde sistema actual

Que cambia

Antes	Despues
`TEAM#backend-mac-1`	`NODE#cesar-macbook-pro`
`--team backend --machine mac-1`	`--node cesar-macbook-pro` (o auto-detect)
`config.py: MACHINES = [...]`	`.covacha-node.yml` por maquina
Roles fijos por maquina	Roles dinamicos por capabilities
2-4 maquinas max	N maquinas (sin limite)
Especializacion hardcodeada	Especializacion inferida

Compatibilidad backward

Los scripts existentes (`claim_task.py`, `release_task.py`, etc.) siguen funcionando:

- Si reciben `--team` y `--machine` → mapean a node_id internamente
- Si reciben `--node` → usan directo
- Deprecation warning en `--team/--machine` por 2 releases

```
# Backward compat en claim_task.py
def parse_node_args(args):
    if args.node:
        return args.node
    elif args.team and args.machine:
        warnings.warn("--team/--machine deprecated. Usa --node", Deprec
        return f"{args.team}-{args.machine}"
    else:
        # Auto-detect from .covacha-node.yml
        return load_node_config()["node_id"]
```

---

11. MCP Server Adaptado

El Covacha MCP Server detecta el nodo automaticamente:

```
{
```

```
  "mcpServers": {
    "covacha": {
      "command": "covacha-mcp-server",
      "env": {}
    }
  }
}
```

Zero config. El server:

1. Lee `~/.covacha-node.yml`
2. Si no existe → ejecuta `covacha init` interactivo
3. Registra el nodo
4. Expone tools con contexto del nodo actual

Las tools MCP saben que nodo eres sin que lo pases como parametro:

```
// Antes:
covacha_claim_task({ task_id: "043", team: "backend", machine: "mac-1"

// Despues:
covacha_claim_task({ task_id: "043" })
// → El server ya sabe que eres "cesar-macbook-pro"
```